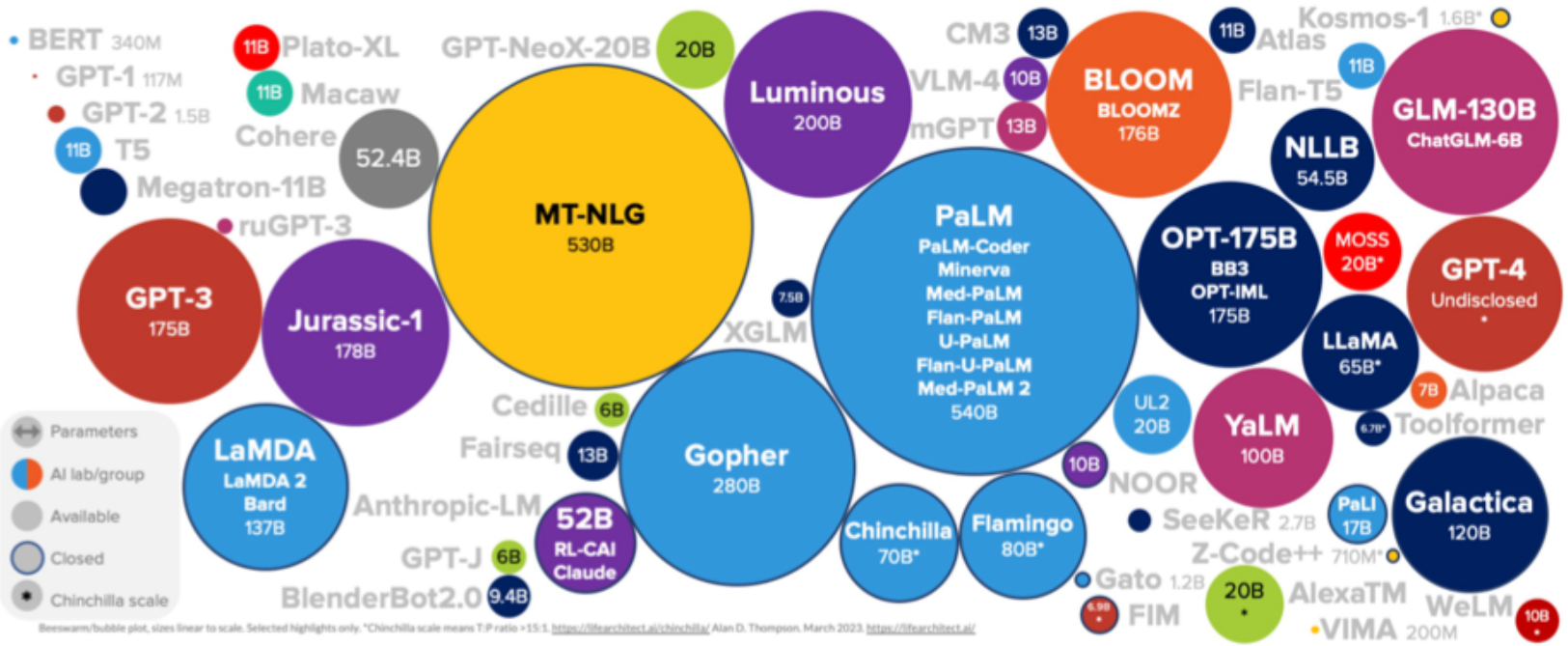
Pre-training
Part 1

Slides mostly borrowed from materials by Jay Alammar, Pawan Goyal,
the Stanford NLP course materials and other online resources

Pre-training

- What is Pretraining?
- Pretrained LSTMs: ELMo
- Pretraining Transformers: BERT and Large Language Models

The idea of transfer learning



Examples of popular pre-trained Language Models

What has made Transformers popular is that they can be combined with the idea of transfer learning.

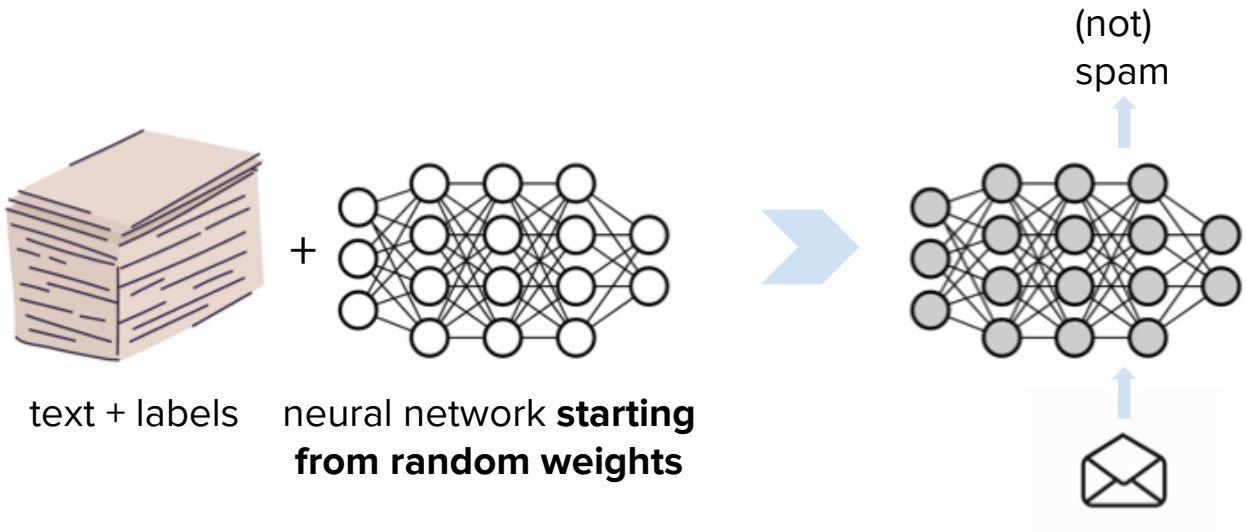
Transformers have become the go-to model for building large pretrained language models which can be adapted for several tasks

https://rycolab.io/classes/llm-s24/

Standard Supervised Deep Learning

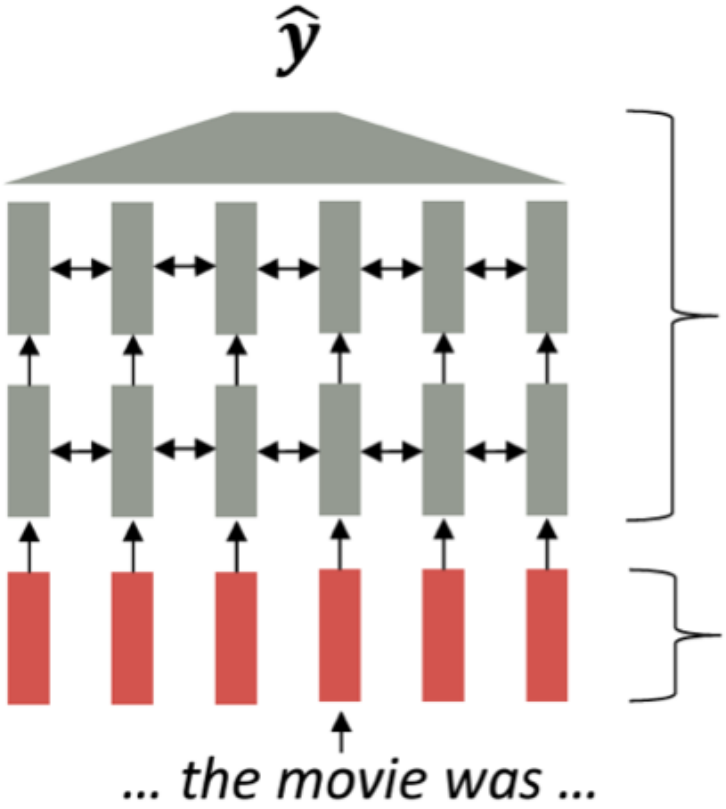
Labeling a sizeable dataset needs a lot of effort / time.

Every new learning problem needs labeling of a new dataset



https://utah-cs6340-nlp.notion.site/Natural-Language-Processing-bd1a2ca290fc44f69556908ad8d25c70

Why is this an issue

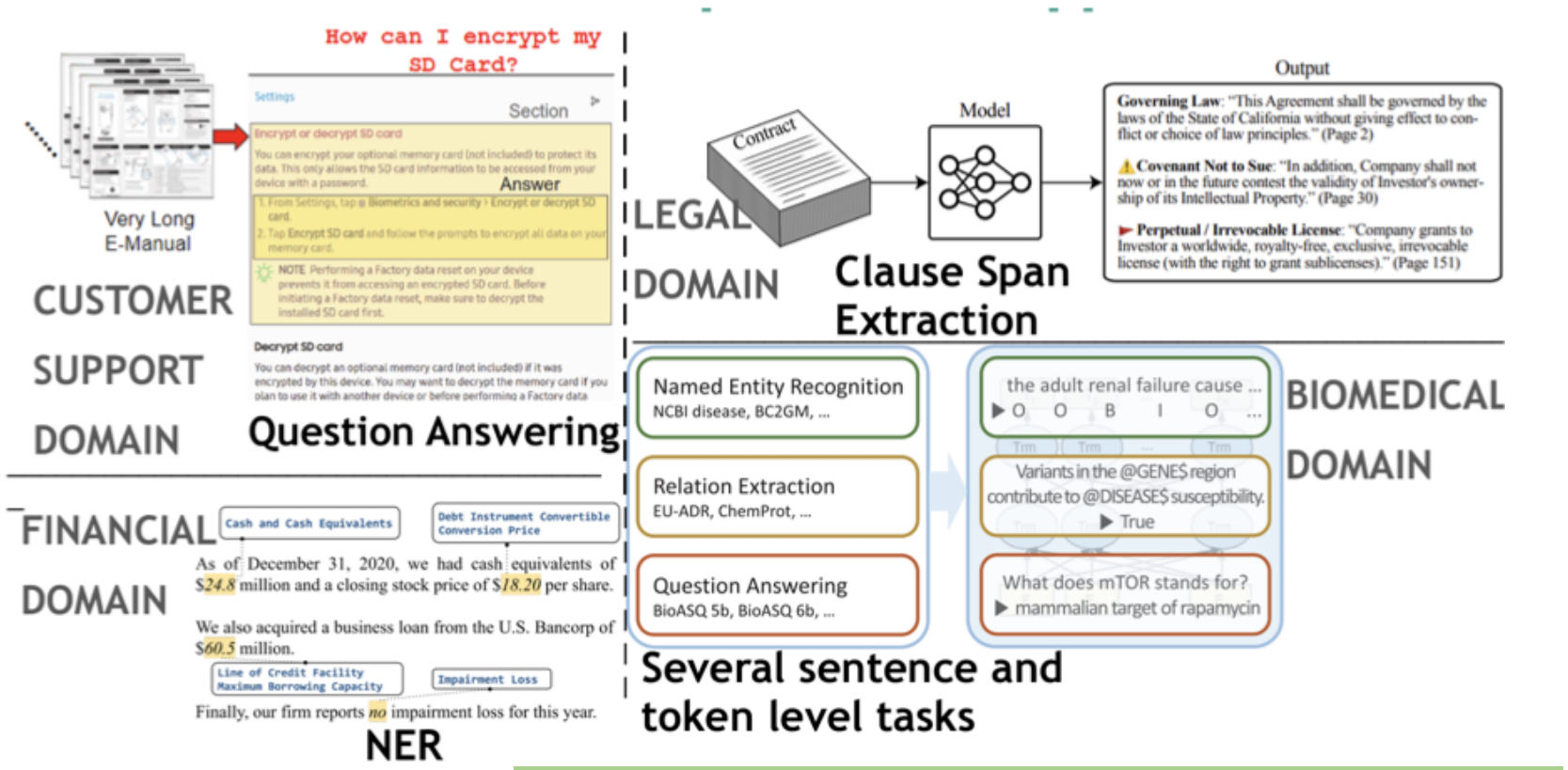


Randomly initialized

Word Vectors

The training data for our downstream task (like question answering) must be sufficient to teach all contextual aspects of language, and train millions of parameters.

Abundance of domain-specific NLP Applications



Very difficult to get sufficient labeled data for each task

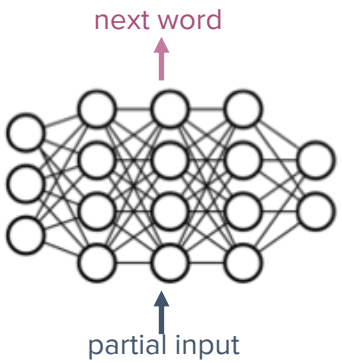
Pretrain-then-Finetune (new paradigm from 2018)

Stage 1:
Pretrain a model

neural network **starting from <u>random</u> weights**

Text
(unlabeled)

+



Self-supervised learning

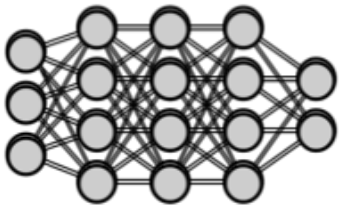
Objective: generate next or masked word
*(does **not** require humans to label the next word)*

Stage 2:
Finetune the model

neural network **starting from <u>pretrained</u> weights**

text + **labels**

+



This phase now needs much smaller labeled data

Objective: standard supervised training

https://utah-cs6340-nlp.notion.site/

Transfer Learning with Language Modeling

Language modelling is the popular choice of pretraining in NLP.

The key idea is to **train a neural network language model (on vast text corpora).** And **then, use it to transfer that knowledge to any target task in NLP** we care about.

This is a **very successful** idea in NLP:

- Supervised systems for various NLP tasks used to require **millions of examples** to learn tasks. NLP systems that use language models as a starting point, need much fewer - **thousands** of examples to do so!

https://rycolab.io/classes/llm-s24/

What does a model learn from pretraining?

- One dog in the front room, and two dogs in the courtyard
- It wasn't just big it was enormous
- The author of "A Room of One's Own" is Virginia Woolf
- The doctor told me that he …
- The capital of France is Paris
- The square root of 4 is 2

Plural forms
Synonyms (big → enormous)
World knowledge - names of persons, locations

Pronoun of a word

Even some mathematics

Note: such pretraining can also lead to learning of wrong concepts, bias, etc.

https://web.stanford.edu/~jurafsky/slp3/

Big idea

- Text contains enormous amounts of world knowledge
- Pretraining on lots of text with all that knowledge is what gives language models their ability to do so much

https://web.stanford.edu/~jurafsky/slp3/

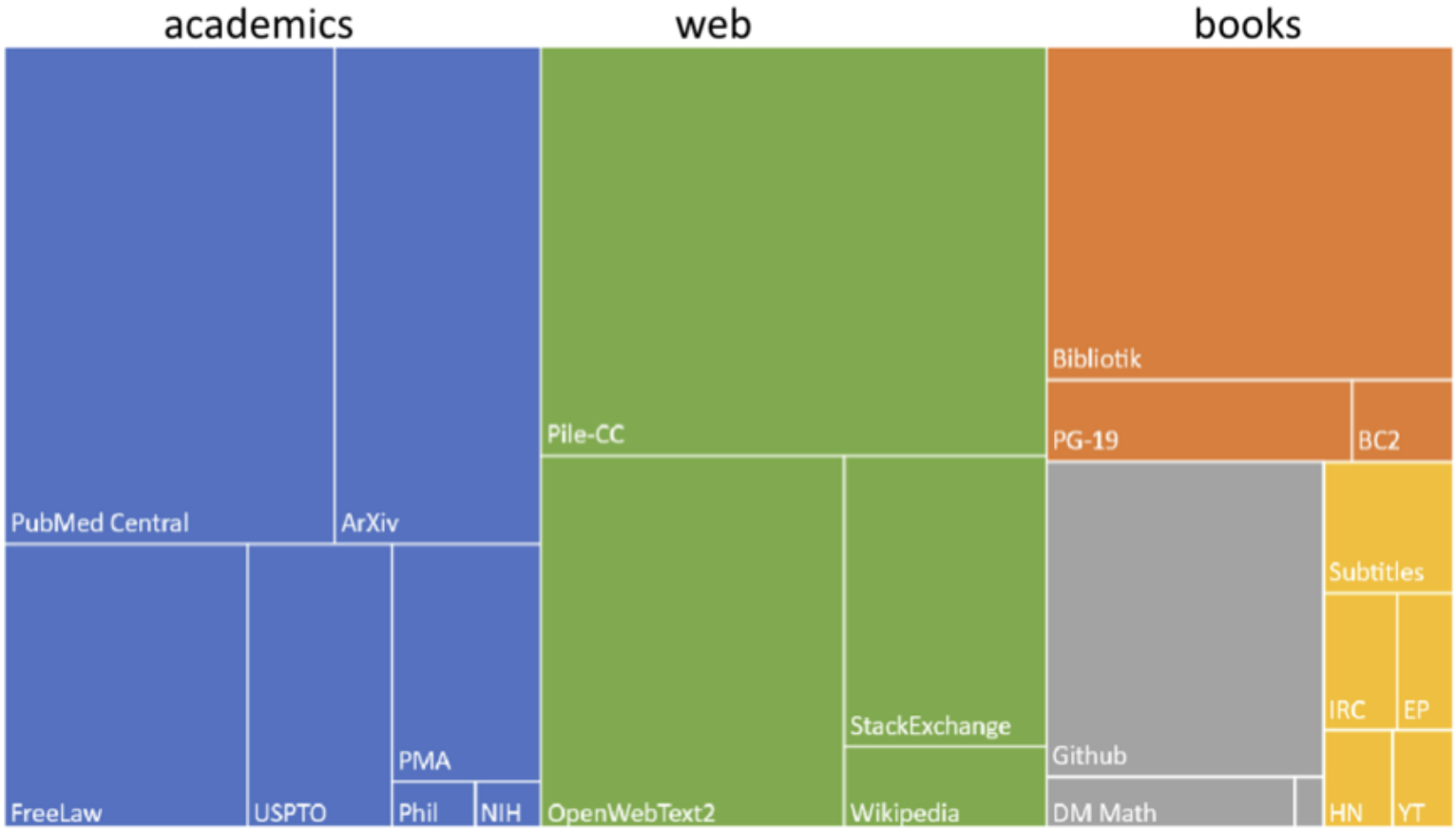
LLMs are mainly trained on the Web

- Common crawl, snapshots of the entire web produced by the non-profit Common Crawl with billions of pages

- Colossal Clean Crawled Corpus (C4; Raffel et al. 2020), 156 billion tokens of English, filtered
  - What's in it? Mostly patent text documents, Wikipedia, and news sites

https://web.stanford.edu/~jurafsky/slp3/

The Pile: another pretraining corpus



dialog

https://web.stanford.edu/~jurafsky/slp3/

Filtering the pre-training data for quality and safety

Quality is subjective

- Need to remove boilerplate text, adult content
- Can pick up various biases inherent in text

Safety is also subjective

- Toxicity detection is important, although that has mixed results
- Can mistakenly flag data written in dialects like African American English

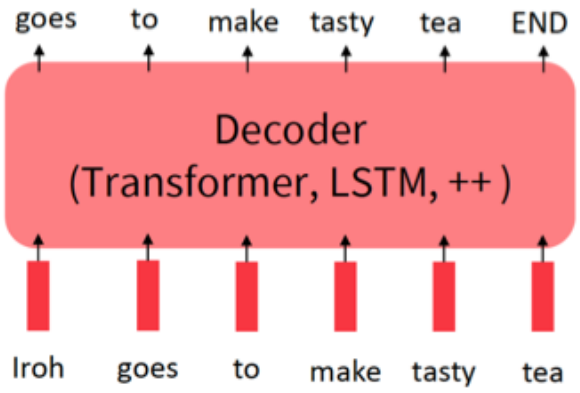
https://web.stanford.edu/~jurafsky/slp3/

The Pretraining / Fine Tuning paradigm

Pretraining can improve NLP applications by serving as parameter initialization.

Step 1: Pretrain (on language modeling)

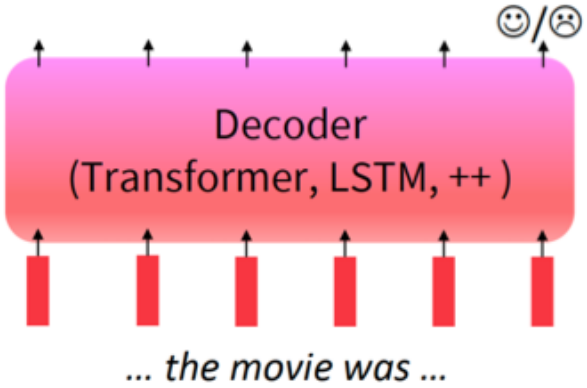
Lots of text; learn general things!



A task like predicting the next word or a masked word (does not need labeling)

Step 2: Finetune (on your task)

Not many labels; adapt to the task!

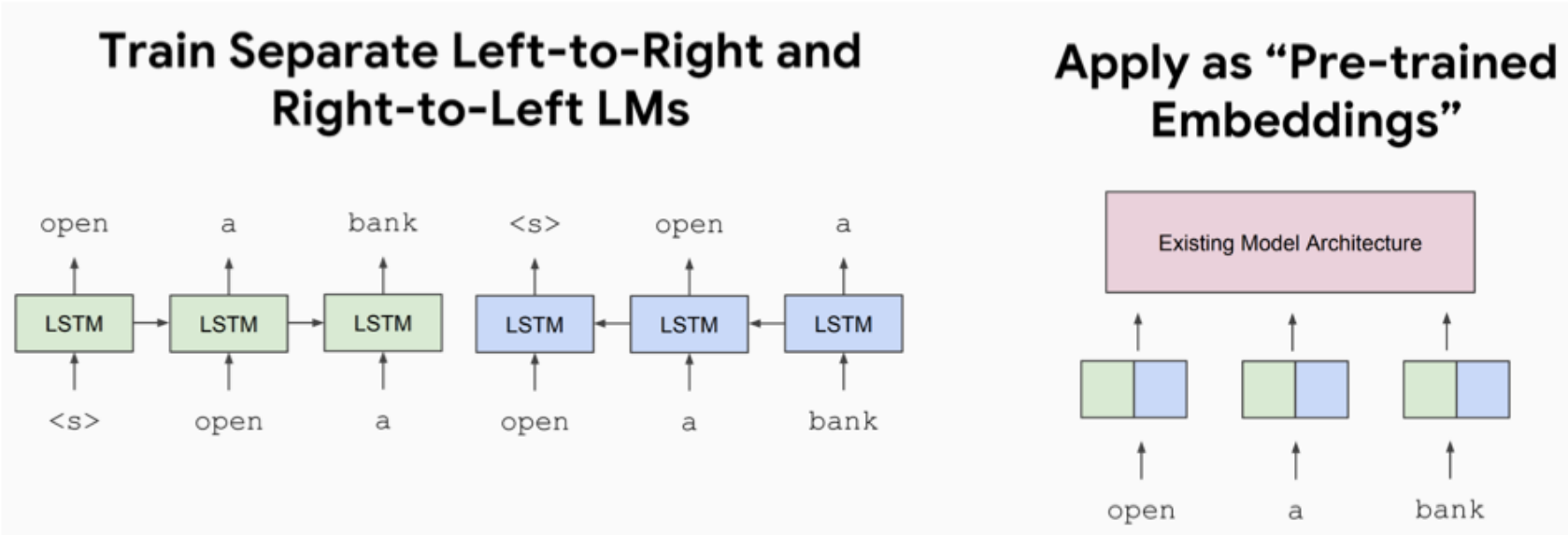


Instead of random initialization of model parameters, start with the pre-trained configuration which already has a lot of world knowledge.

https://web.stanford.edu/class/cs224n/

Pre-trained LSTMs: ELMo

Pretrained LSTMs: ELMo

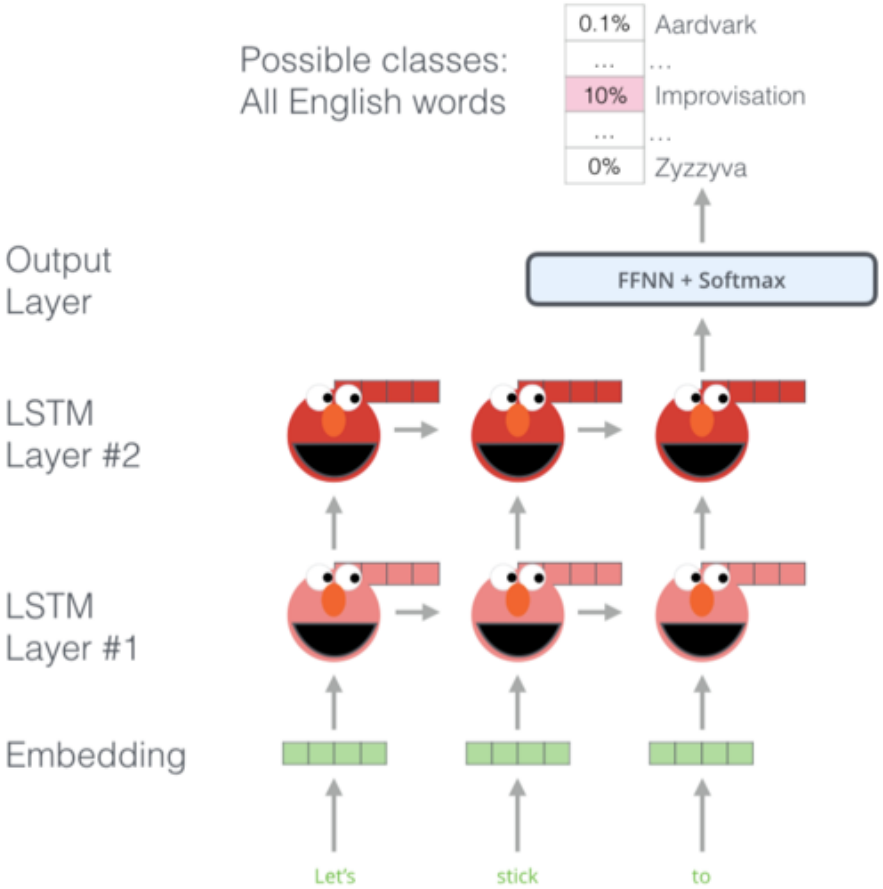


Pre-trained 2-layer LSTMs, both forward and backward, over a large corpus

The LSTM hidden states that we get (for different words) after the pre-training represents world knowledge. Use these hidden state vectors for new tasks.

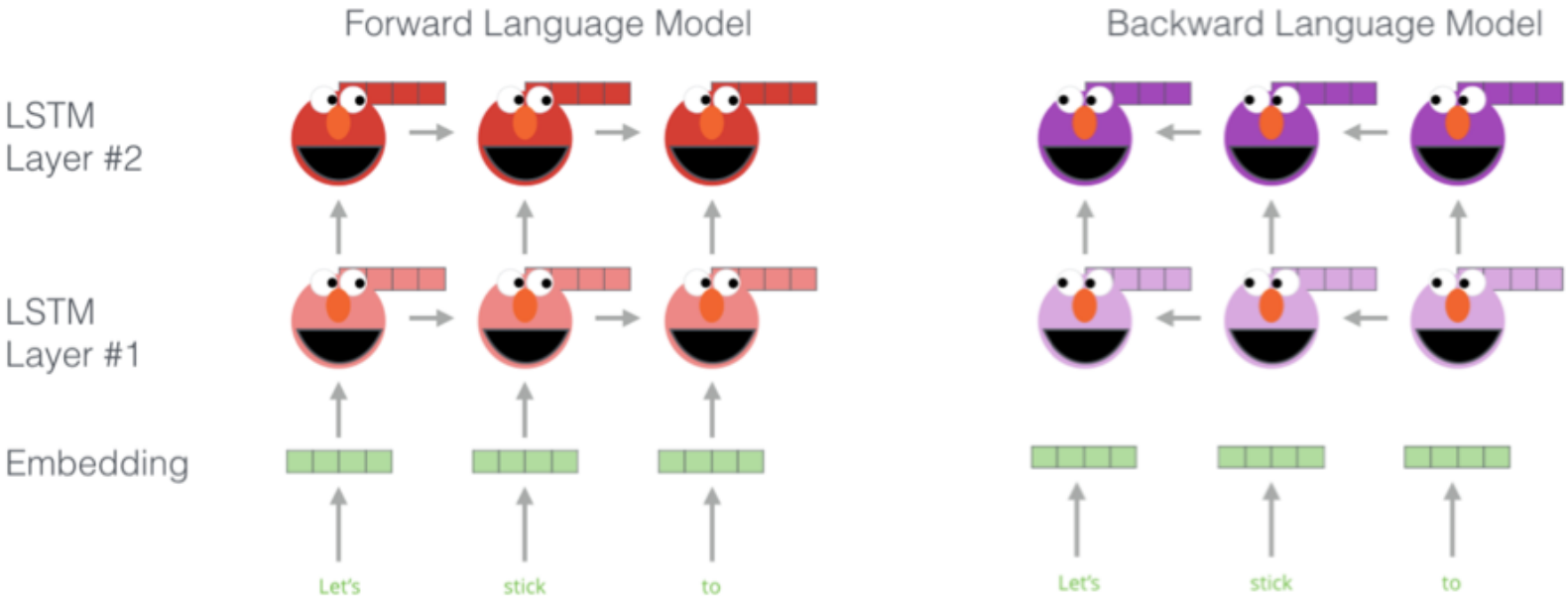
https://rycolab.io/classes/llm-s24/

ELMo: Training a LM (2-layer LSTM)



https://jalammar.github.io/illustrated-bert/

A bidirectional LM



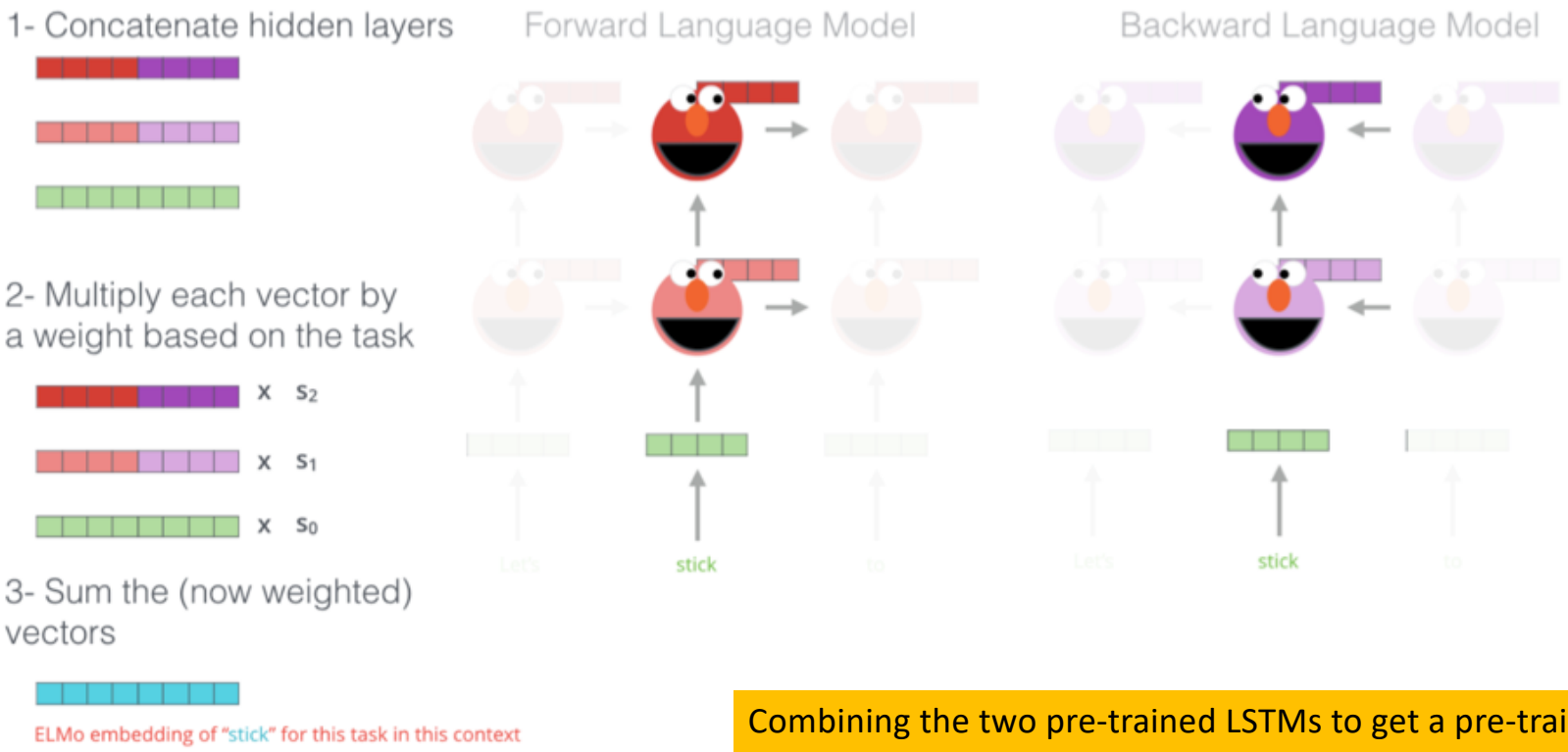
Predicting the next word

Predicting the previous word

The two LSTMs are trained independently

https://jalammar.github.io/illustrated-bert/

ElMo: Obtaining the final embeddings



Combining the two pre-trained LSTMs to get a pre-trained embedding for a word (which will be used for the downstream task). Used the representations of a word at different layers.

https://jalammar.github.io/illustrated-bert/

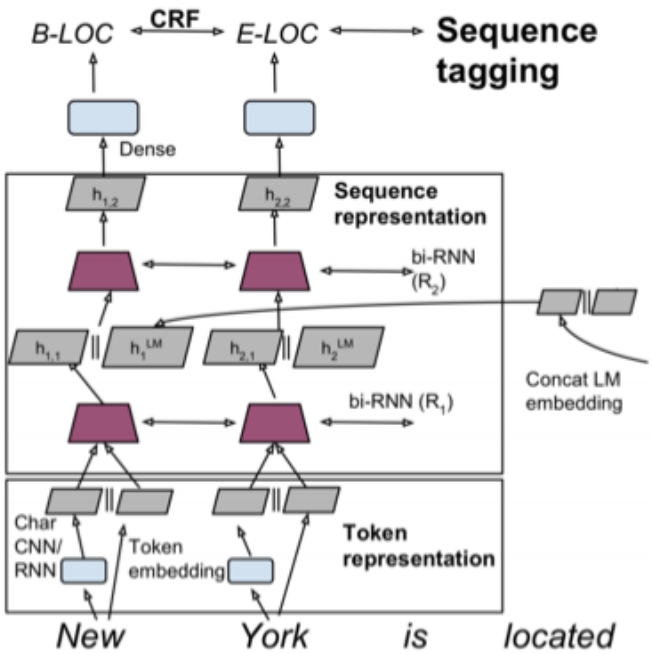
ELMo: Weighting of layers

The two biLSTM NLM layers have differentiated uses/meanings

- Lower layer is better for lower-level syntax, etc. $\rightarrow$ Part-of-speech tagging, syntactic dependencies, NER
- Higher layer is better for higher-level semantics $\rightarrow$ Sentiment, Semantic role labeling, question answering, SNLI

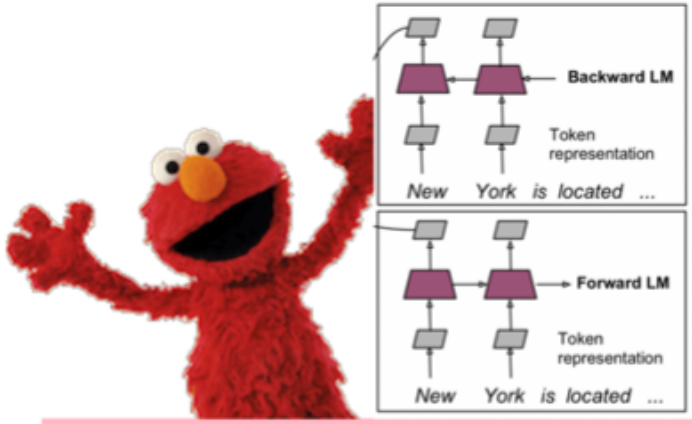
Which layer to give more weightage to?
This depends on the particular downstream task

Using ELMo for a task (sequence tagging; NER)



$$\mathbf{h}_{k,1} = [\overrightarrow{\mathbf{h}}_{k,1}; \overleftarrow{\mathbf{h}}_{k,1}; \mathbf{h}_k^{LM}].$$

https://web.stanford.edu/class/cs224n/



ELMo representation:
A deep bidirectional neural LM

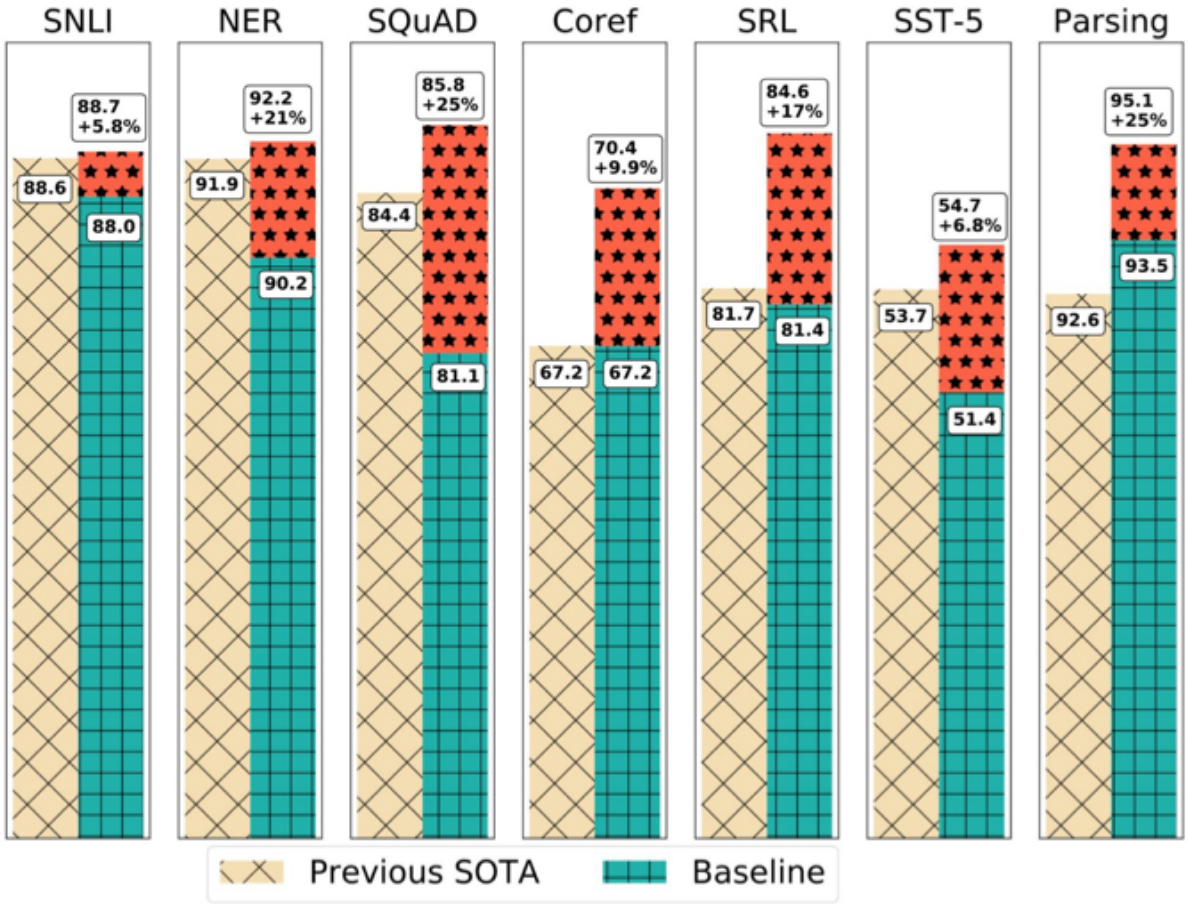
Use learned, task-weighted average of (2) hidden layers

Say, we are training a bi-RNN for sequence tagging / NER task.

Embeddings from the pre-trained ELMo are appended to the hidden states of the bi-RNN.

Pre-trained ELMo embeddings are frozen. The loss is used to update only the parameters of the smaller bi-RNN.

ELMo: Results



https://rycolab.io/classes/llm-s24/

Possibly the first paper that showed large improvements over multiple tasks, using the same model.

Previous SOTA for different tasks were different models.

Demonstrated the power of knowledge transfer using pre-trained Language Models.

ElMo - How does the pretraining help with efficient fine-tuning?

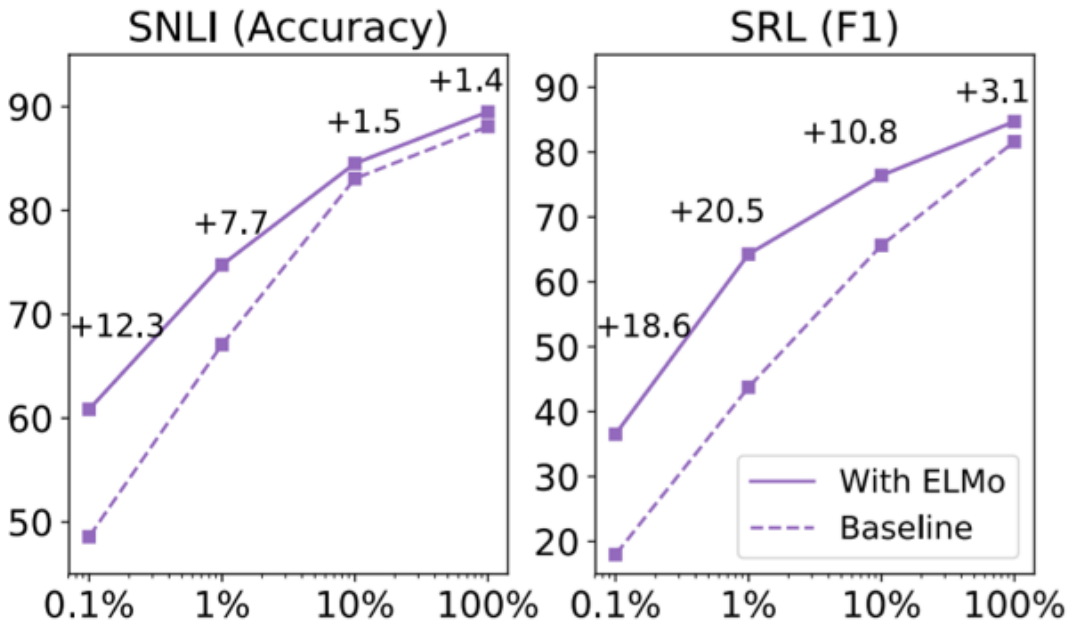


Figure 1: Comparison of baseline vs. ELMo performance for SNLI and SRL as the training set size is varied from 0.1% to 100%.

X-axis: amount of training data
Y-axis: performance

Using the pre-trained model is especially better when the amount of training data is small.